UNIVERSIDAD TECNOLÓGICA NACIONAL

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

TRABAJO PRÁCTICO INTEGRADOR

FOOD STORE

Sistema de gestión de pedidos de comida

Materia: Programación II

Estudiantes: Valentino Nazar y Agustín Estrella

Índice

• 1. Introducción — 1

• 2. Marco teórico — 2

• 3. Modelo orientado a objetos — 2

• 4. Arquitectura y decisiones técnicas — 2

• 5. Reglas de negocio y validaciones — 3

• 6. Pruebas y capturas del sistema — 3

• 7. Dificultades y soluciones — 7

• 8. Bibliografía y enlaces — 7

Sistema de gestión de pedidos de comida

1. Introducción 1

La numeración corresponde a las páginas de este informe.

1. Introducción

Food Store es una aplicación de consola para administrar un negocio de comidas. Permite gestionar categorías, productos, usuarios y pedidos mediante operaciones de alta, baja lógica, modificación y consulta. El sistema diferencia los permisos de administradores y clientes a partir de un inicio de sesión.

La información se conserva en colecciones mientras la aplicación está en ejecución. Al cerrar el programa, los datos se pierden, de acuerdo con el alcance definido en la consigna.

2. Marco teórico

Programación orientada a objetos: organiza el programa en clases y objetos que agrupan datos y comportamientos.

Clase abstracta: Base es una clase abstracta que concentra identificador, estado de baja y fecha de creación. Calculable define el método abstracto para calcular el total de un pedido.

Herencia: Categoría, Producto, Usuario, Pedido y DetallePedido heredan de Base y reutilizan sus atributos y comportamiento.

Encapsulamiento: los atributos se mantienen privados y se accede a ellos mediante properties. Los setters validan los valores que pueden modificarse.

Polimorfismo: Menu define el método abstracto mostrar(). MenuAdministrador y MenuCliente implementan el comportamiento según el rol.

Composición y agregación: Pedido crea y administra sus DetallePedido, que forman parte de su ciclo de vida. Categoría se relaciona con varios productos, mientras cada producto pertenece a una categoría.

Excepciones: las excepciónes propias permiten informar errores del dominio, como entidades inexistentes, datos inválidos, duplicados y stock insuficiente.

Colecciones: las entidades se almacenan en listas privadas de los servicios, que realizan las búsquedas y operaciones de negocio.

Python y type hints: Python es un lenguaje de alto nivel y multipropósito. Los type hints documentan los tipos esperados en atributos, parámetros y retornos, facilitando la lectura y el mantenimiento del código.

3. Modelo orientado a objetos

El modelo implementa las entidades indicadas en el diagrama UML: Base, Categoría, Producto, Usuario, Pedido y DetallePedido. También incorpora los enumerados Rol, Estado y FormaPago, y la interfaz Calculable.

El identificador se genera desde un contador compartido en Base. La fecha de creación se asigna automáticamente. Pedido es responsable de crear los detalles y expone una tupla para impedir modificaciones externas de la colección.

4. Arquitectura y decisiones técnicas

El proyecto se separa en tres capas. La consola solicita y valida entradas, los servicios mantienen las colecciones y aplican las reglas de negocio, y las entidades representan el modelo y protegen su propio estado. La dependencia se mantiene en el sentido consola → servicios → entidades.

Se utiliza solamente la biblioteca estándar de Python. main.py crea los servicios, carga un administrador y datos de ejemplo, y controla los intentos de inicio de sesión. Las reglas de negocio se encuentran en los servicios y las entidades, no en main.py.

5. Reglas de negocio y validaciónes

Los precios y el stock no pueden ser negativos.

Los pedidos requieren un usuario activo y cantidades mayores a cero.

La cantidad solicitada no puede superar el stock disponible.

Un pedido no repite productos: al agregar el mismo producto se acumula la cantidad.

Los mails de usuarios y los nombres de categorías se validan para evitar duplicados.

No se puede crear un producto con una categoría que no pertenezca a la colección de categoríasactivas.

No se puede eliminar una categoría que tenga productos activos.

Los estados siguen las transiciones permitidas: PENDIENTE a CONFIRMADO o CANCELADO;CONFIRMADO a TERMINADO o CANCELADO.

Al cancelar o eliminar un pedido no terminado se devuelve el stock.

Las bajas son lógicas y los objetos permanecen en las colecciones.

El cliente solo consulta categorías, productos disponibles, sus pedidos y sus propios datos.

6. Pruebas y capturas del sistema

El archivo pruebas.py ejecuta comprobaciones de altas, modificaciones, duplicados, validaciónes de datos, autenticación, control de stock, cálculo de totales, transiciones de estado, cancelación, eliminación lógica y devolución de stock. La ejecución finaliza con todos los casos incluidos aprobados.

Ejecución de pruebas

Inicio de sesión y menu de administrador

Listado de productos

Listado de categorías

Listado de pedidos

menú del cliente

7. Dificultades y soluciones

Una dificultad central del dominio es mantener consistente el stock cuando se crea, cancela o elimina un pedido. Para evitar descuentos parciales, el servicio primero construye el pedido y valida todos sus detalles; solo cuando todos son válidos descuenta el stock y agrega el pedido a la colección. Si ocurre una excepción durante la validación, no se modifica el stock ni se guarda el pedido.

Otra decisión importante es impedir que el cliente acceda a operaciones de modificación. Se resolvió mediante una clase Menu abstracta y dos implementaciones concretas, una para el administrador y otra para el cliente. Las validaciónes de datos se distribuyeron entre setters, servicios y lectura de consola según la responsabilidad de cada capa.

8. Bibliografía y enlaces

Python Software Foundation. Documentación oficial de Python: https://docs.python.org/3/

Python Software Foundation. Módulo abc: https://docs.python.org/3/library/abc.html

Python Software Foundation. Módulo enum: https://docs.python.org/3/library/enum.html Python Software Foundation. Módulo datetime: https://docs.python.org/3/library/datetime.html

Repositorio GitHub: pendiente de publicar y agregar enlace.

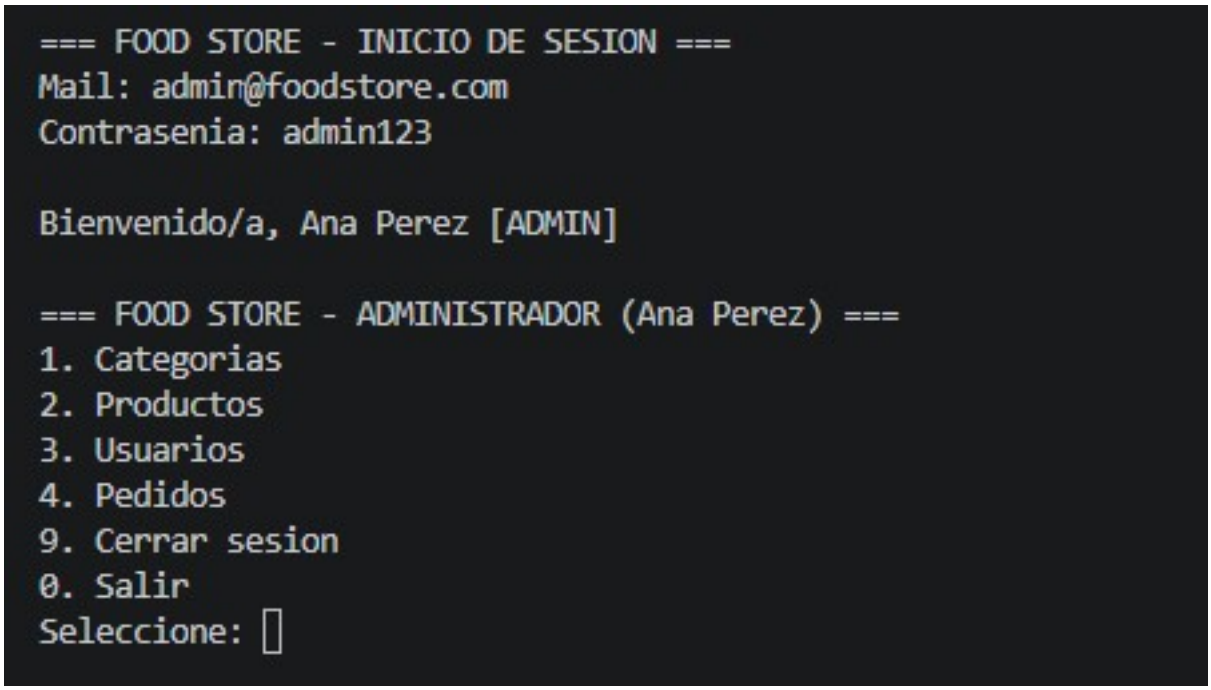
Video demostrativo: Ver video demostrativo en Google Drive

Segundo integrante: Valentino Nazar.

El repositorio GitHub se completará cuando esté publicado. El video demostrativo se encuentra disponible mediante el enlace anterior.

Capturas del sistema





```
--- PRODUCTOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
ID: 4 | Hamburguesa clasica | Precio: $4500.00 | Stock: 10 | Categoria: Hamburguesas
ID: 5 | Hamburguesa doble | Precio: $6000.00 | Stock: 5 | Categoria: Hamburguesas

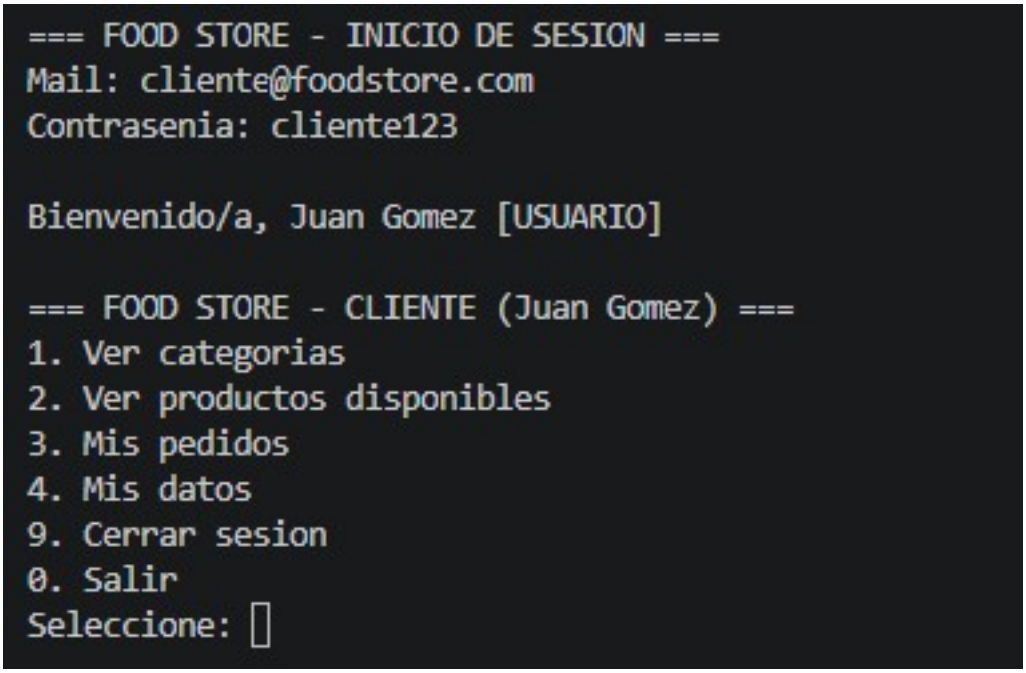
--- PRODUCTOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione:
```

```
--- CATEGORIAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
ID: 3 | Hamburguesas | Hamburguesas y combos

--- CATEGORIAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione:
```

```
--- PEDIDOS ---
1. Listar
2. Crear
3. Actualizar estado o forma de pago
4. Eliminar
0. Volver
Seleccione: 1
ID: 6 | Ana Perez | PENDIENTE | TRANSFERENCIA | Total: $4500.00 | 2026-11-04

--- PEDIDOS ---
1. Listar
2. Crear
3. Actualizar estado o forma de pago
4. Eliminar
0. Volver
Seleccione:
```


=== FOOD STORE - INICIO DE SESION ===
Mail: cliente@foodstore.com
Contrasenia: cliente123
Bienvenido/a, Juan Gomez [USUARIO]
=== FOOD STORE - CLIENTE (Juan Gomez) ===
1. Ver categorias
2. Ver productos disponibles
3. Mis pedidos
4. Mis datos
9. Cerrar sesion
0. Salir
Seleccione: